

Intern Coding Challenge: Event Booking System

Objective:

You are required to build an Event Booking System consisting of a FastAPI backend, ReactJS frontend, and Python SDK, with automation scripts.

Your solution should be fully functional, adhere to OpenAPI standards, and demonstrate best practices in backend, frontend, database, and SDK development.

Tasks & Requirements

1. Backend Development (FastAPI & SQLite)

Implement a FastAPI service with the following endpoints:

- **POST** /events/ → Create a new event.
- **GET** /events/ → Retrieve all events.
- **GET** /events/{event_id} → Retrieve a specific event.
- **POST** /events/{event_id}/book → Book a seat for an event.
- **DELETE** /bookings/{booking_id} → Cancel a booking.

Use SQLite as the database.

Ensure the API follows OpenAPI standards (Swagger UI should be accessible).

Implement error handling (appropriate 4xx/5xx).

Use dependency injection for database operations.

Write unit tests for the backend.

🚩 Trick Logic in Backend (Hidden Complexity)

- Each event has a limited number of seats.
- Do not allow booking more than available seats.
- If fully booked, reject new bookings.
- Cancellations free up seats.

****Your implementation should correctly enforce the rules above.****

****🔴 Examples:****

- ✅ Event A has 5 seats → 5 bookings allowed
- ❌ 6th person tries to book → error
- ✅ On cancel, 1 seat becomes available

2. Database (SQLite)

Create the following database tables:

...

```

CREATE TABLE events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    description TEXT,
    total_seats INTEGER NOT NULL,
    available_seats INTEGER NOT NULL CHECK (available_seats >= 0),
    date TIMESTAMP NOT NULL
);
\q

...

CREATE TABLE bookings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    event_id INTEGER NOT NULL,
    user_name TEXT NOT NULL,
    booking_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE
);
\q

```

Use Alembic for database migrations.

Provide a sample SQL file (`seed\_data.sql`) to populate initial test data.

3. Frontend Client (ReactJS)

Develop a ReactJS-based frontend.

Use Axios for making API calls.

The frontend should allow users to:

- View all events with availability
- Book a seat; cancel a booking
- See available seats in near real time

Ensure the frontend interacts only via API calls (no direct database access).

Style is not a priority, but functionality must work correctly.

**** 🚀 Bonus:**** Real-time updates where meaningful.

4. Platform SDK (Generated via OpenAPI Generator CLI)

Instead of manually writing an SDK, generate a Python SDK using the OpenAPI Generator CLI. This SDK should allow interaction with the API.

****Instructions to Generate SDK****

```

```bash
npm install -g @openapitools/openapi-generator-cli
openapi-generator-cli generate -i http://localhost:8000/openapi.json -g
python -o event_sdk

```

...

**\*\*Expected SDK Usage (After Generation & Testing):\*\***

```
```python
from event_sdk.api.events_api import EventsApi
from event_sdk import ApiClient

client = ApiClient()
api = EventsApi(client)

# Example usage
# result = api.<your_method_here>()
# print(result)
```
```

---

### 5. Development Setup Script (`setupdev.bat`)

Create a Windows batch script to set up the environment:

```
```bat
@echo off
echo Setting up backend...
python -m venv env
call env\Scripts\activate
pip install -r requirements.txt
alembic upgrade head

echo Setting up frontend...
cd frontend
npm install
cd ..

echo Setup complete.
```
```

---

### 6. Execution Script (`runapplication.bat`)

Create a Windows batch script to start the full application.

```
```bat
@echo off
echo Starting backend...
call env\Scripts\activate
python main.py

echo Starting frontend...
cd frontend
npm start
```
```

---

## Evaluation Criteria

- ✅ Code Quality: Well-structured, clean, and modular code.
- ✅ Correct API Implementation: API should work correctly and follow OpenAPI standards.
- ✅ Proper Error Handling: Handles bad inputs and errors gracefully.
- ✅ Platform SDK Generation: SDK must be generated correctly using OpenAPI Generator CLI.
- ✅ Frontend Integration: The ReactJS frontend should correctly communicate with the backend.
- ✅ Automation Scripts: setupdev.bat and runapplication.bat should work as expected.
- ✅ Unit Tests: Backend should include test cases.
- ✅ Documentation: Include a README with installation and execution steps.
- ✅ Backend Trick Logic: Ensure candidates correctly implement the specified rules.

---

## ## Deliverables

- Backend (FastAPI) source code
- SQLite database & migration files
- ReactJS frontend code
- Python SDK (generated via OpenAPI Generator CLI)
- Setup & Execution Scripts (`setupdev.bat`, `runapplication.bat`)
- Unit tests for backend
- README explaining how to set up and run the project

🕒 **\*\*Time Limit:\*\*** 2-3 Days

---

## ## 💡 Additional Notes

- You are **\*\*NOT\*\*** allowed to use Docker.
- Ensure proper API validation and error handling.
- The frontend does **\*\*not\*\*** need to be fancy - focus on functionality.
- Bonus points for implementing extra features like charts or real-time updates.